

1. Executive Summary

SECURITY SCORE

10

CRITICAL

0

HIGH

11

MEDIUM

11

INFO/LOW

5

1.2 ISMS-P 컴플라이언스 위반 요약

이번 분석에서 총 **36**건의 ISMS-P 인증기준 위반 또는 위반 의심 사례가 탐지되었습니다. 관련 부서는 본 리포트의 세부 항목을 검토 후 조치 요망.

#	ISMS-P 위반 핵심 내용	관련 취약점	심각도
1	본 코드만으로는 CSRF에 의한 주요 변경 요청 보호 미흡(참조 1의 CSRF 점검 ...	HTTP 메서드 미지정 RequestMapping(화면 조회용)	LOW
2	직접적인 취약점으로 확정되진 않지만, 운영 비밀정보를 소스에 포함하지 말아야 한다는 ...	하드코딩된 더미 JWT 문자열로 인한 비밀정보 탐지	LOW
3	ISMS-P 보호대책 중 2.5 인증 및 권한관리, 2.6 접근통제 원칙에 어긋날 수...	Docker 컨테이너가 non-root USER 없이 실행되어 root 권한 남용 가능	HIGH
4	ISMS-P 2.2.6 보안 위반 시 조치: 컨테이너 실행 권한을 최소 권한으로 제한하...	Docker 컨테이너가 non-root USER 없이 실행되어 root 권한으로 동작할 수 있음	HIGH

5	ISMS-P 관점에서는 안전한 설정 미흡이 운영상 보안통제 강화 필요사항에 해당할 수...	Kubernetes 컨테이너 권한 상승 방지 설정 미흡	MEDIUM
6	ISMS-P의 보안 구성 관리 및 최소권한 원칙에 부합하지 않습니다. 컨테이너 실행 ...	Kubernetes 컨테이너 권한 상승 방지 설정 미흡	MEDIUM
7	ISMS-P 보안 구성 및 하드닝 미흡으로 인한 정보보호 통제 부재에 해당합니다. 특...	Kubernetes 컨테이너 비루트 실행 강제 설정 누락	INFO
8	주요정보통신기반시설 기술적 취약점 분석·평가 방법 상세가이드의 취지상, 컨테이너/서버...	Kubernetes 컨테이너에 privilege escalation 방지 보안컨텍스트 미설정	MEDIUM
9	ISMS-P 관점에서는 직접적인 개인정보 처리 로직 위반(예: 3.1.1 개인정보 수...	Kubernetes 컨테이너 비루트 실행 설정(runAsNonRoot) 누락 권고	INFO
10	ISMS-P의 최소권한 원칙 및 접근통제/보안설정 요구사항에 부합하지 않는 구성입니다...	Kubernetes Pod/컨테이너의 비루트 실행 보장 미설정	INFO
11	ISMS-P 관점에서 최소권한 원칙 및 안전한 운영 설정 준수 미흡 가능성이 있으나,...	Kubernetes 컨테이너 비루트 실행 보장 설정 누락	INFO
12	ISMS-P 관점에서 개인정보 처리 시스템의 실행 환경에 대한 최소권한 원칙 및 안전...	Kubernetes Pod에 비루트 실행 보안 설정 누락 경고	INFO
13	ISMS-P 기술적 취약점 관점에서 컨테이너 실행 계정 및 권한 제한 미비는 최소권한...	Kubernetes 컨테이너 권한 상승 방지 설정 누락 여부 검증 필요	MEDIUM
14	ISMS-P 2.6 접근통제 및 2.10 시스템 및 서비스 보안관리 관점에서 컨테이너...	Kubernetes 컨테이너 권한 상승 방지 설정 누락	MEDIUM
15	ISMS-P 기술적 취약점 관리 관점에서 컨테이너 권한 최소화 및 접근통제 설정이 미...	Kubernetes 컨테이너 권한 상승 방지 설정 미흡	MEDIUM
16	ISMS-P 기술적 보호조치 관점에서 안전한 실행 환경 및 최소권한 원칙을 충족하지 ...	Kubernetes 백엔드 컨테이너가 비루트 실행 보장을 명시하지 않음	INFO
17	직접적인 ISMS-P 웹 취약점보다는 Kubernetes 런타임 보안 설정 미흡에 해...	Kubernetes 컨테이너 non-root 실행 설정 누락 여부 검토 필요	INFO

18	ISMS-P 2.10.2 클라우드 보안 및 2.11.2 취약점 점검 및 조치 관점에서...	Kubernetes 컨테이너 권한 상승 방지 설정 미흡	MEDIUM
19	권한 최소화 및 안전한 실행 환경 통제 미흡에 해당하며, ISMS-P의 접근통제/시스...	Kubernetes 컨테이너에 비루트 실행 보안 설정 미흡	INFO
20	ISMS-P 관점에서 컨테이너 실행 권한을 최소화하지 않아 기술적 취약점 관리가 미흡...	Kubernetes 컨테이너 권한 상승 방지 설정 미흡	MEDIUM
21	ISMS-P 기술적 취약점 관점에서 안전하지 않은 컨테이너 실행 설정이며, 최소권한 ...	Kubernetes nginx 컨테이너의 privilege escalation 차단 설정 누락	MEDIUM
22	현재 코드만으로 ISMS-P 위반 확정 불가. 다만 세션/토큰 기반의 중앙 집중적 접근...	단독 코드 기준으로는 실제 비즈니스 로직 취약점 확정 불가	INFO
23	ISMS-P의 최소권한 원칙 및 세밀한 접근제어 요구에 위배될 수 있으며, 제공된 참...	외부 API 키 및 모델 파라미터를 클라이언트 입력에 의존하는 정책 우회 가능성	MEDIUM
24	ISMS-P 웹 취약점 분석 항목 18. 쿠키 변조/임의 사용자 권한 탈취 관점의 권...	클라이언트 저장 userId를 신뢰하는 비밀번호 변경 요청으로 인한 권한 우회(IDOR) 가능성	HIGH
25	참조 3의 웹 애플리케이션 권한 검증 미흡(인증이 필요한 주요 정보/기능에 대해 요청...	작품 등록 API의 userId를 클라이언트가 직접 지정하는 권한 우회 위험	HIGH
26	참조 3의 비즈니스 로직 흐름 통제 및 데이터 무결성 관점의 보완 필요. 반복 제출로...	표지 생성/작품 등록의 중복 제출 방지 및 멍등성 통제가 보이지 않음	LOW
27	ISMS-P 2.7 암호화 적용 위반 가능성: 비밀번호 찾기/변경 과정에서 민감정보가...	임시 비밀번호를 API 응답과 화면에 평문으로 노출	MEDIUM
28	ISMS-P 웹 취약점 중 권한 검증 미흡으로 인한 접근통제 위반에 해당합니다. 제로...	작품 삭제 API의 소유권 검증 부재로 인한 IDOR 가능성	HIGH
29	ISMS-P의 접근통제 및 최소권한 원칙 위반 가능성이 있으며, 제로트러스트 원칙 중...	작품 수정 API의 클라이언트 제공 userId 신뢰로 인한 IDOR 가능성	HIGH
30	ISMS-P 2.6.1(접근통제) 관점에서 사용자 식별정보를 기준으로 한 권한 검증이...	비밀번호 변경 API에서 세션 사용자 검증 누락으로 계정 탈취성 권한우회 가능	HIGH

31	ISMS-P 2.7(암호화 적용) 및 계정관리 관점에서 인증정보(임시 비밀번호)를 응...	ID 찾기/비밀번호 찾기 기능에서 계정 열거 및 임시 비밀번호 평문 노출	HIGH
32	ISMS-P DBMS 계정 관리(D-08) 및 Unix 계정 관리(U-13) 취지에 ...	비밀번호를 평문 비교 조건으로 사용하는 로그인 인증 결함	HIGH
33	ISMS-P 웹 애플리케이션 불충분한 세션 관리(16) 취지 위반 가능성: 세션 ID...	세션 생성·만료 정책이 보이지 않는 불충분한 세션 관리 가능성	MEDIUM
34	ISMS-P 2.7 암호화 적용: 개인정보 및 주요정보의 저장·전달 시 안전한 암호화...	비밀번호 재설정 로직의 자격 검증 부족으로 인한 계정 탈취 가능성	HIGH
35	ISMS-P 2.7 암호화 적용(비밀번호 처리) 및 접근통제 원칙 위반 소지. 안내서...	비밀번호 변경 로직의 사용자 소유권 검증 부재로 인한 권한 우회 가능성	HIGH
36	ISMS-P 관점에서 계정정보 무결성 및 접근통제 실패로 이어질 수 있으며, 중복 계...	회원가입 이메일 중복검사와 저장 사이 원자성 부족으로 중복 계정 생성 가능	MEDIUM

2. Detailed Findings

2.1 코드 정적 분석 결과

1. Docker 컨테이너가 non-root USER 없이 실행되어 root 권한 남용 가능

| **Status:** TP | **Rule:** dockerfile.security.missing-user-entrypoint.missing-user-entrypoint | **File:** mini_project5\book\Dockerfile

제공된 Dockerfile 스니펫에는 ENTRYPOINT만 존재하고 USER 지시어가 없습니다. 이 경우 컨테이너 내부 프로세스는 기본적으로 root 권한으로 실행될 가능성이 높아, 애플리케이션 취약점이 존재할 때 컨테이너 장악 및 호스트/클러스터 측 권한 확대의 발판이 될 수 있습니다. 현재 문맥상 non-root 전환이나 방어 로직이 확인되지 않아 Semgrep 결과는 진탐으로 판단됩니다.

공격 시나리오: 공격자가 애플리케이션 취약점(예: 업로드/경로조작/RCE)을 통해 컨테이너 내부 코드 실행을 얻으면, 프로세스가 root로 동작하는 환경에서는 /app 하위 파일 변조, 민감 설정 접근, 컨테이너 탈출 시도 등 권한 영향이 커집니다. 특히 Kubernetes/EKS 환경에서는 잘못된 볼륨 마운트나 호스트 접근이 결합될 경우 클러스터 측 피해로 이어질 수 있습니다.

취약 코드:

```
FROM public.ecr.aws/amazoncorretto/amazoncorretto:17-al2-jdk
WORKDIR /app
COPY build/libs/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

수정 코드:

```
FROM public.ecr.aws/amazoncorretto/amazoncorretto:17-al2-jdk
WORKDIR /app
COPY build/libs/*.jar app.jar
RUN useradd -r -u 10001 appuser && chown -R appuser:appuser /app
USER 10001:10001
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

수정 가이드: 컨테이너 실행 전용 non-root 사용자를 생성하고, 애플리케이션 디렉터리 소유권을 부여한 뒤 USER를 명시적으로 전환해야 합니다. 또한 멀티 스테이지 빌드/실행 분리, 최소 권한 원칙 적용, 필요 시 읽기 전용 루트 파일

시스템과 추가 capability 제거를 함께 적용하는 것이 바람직합니다.

ISMS-P: ISMS-P 보호대책 중 2.5 인증 및 권한관리, 2.6 접근통제 원칙에 어긋날 수 있으며, OT/제로트러스트 관점에서도 최소 권한(least privilege)과 지속적 검증 원칙에 부합하지 않습니다. 제공된 제로트러스트 안내서의 ZT 정책 수립/지속 모니터링 단계와 비교하면, 실행 주체를 root로 두는 것은 보호 표면을 불필요하게 확대합니다.

AI Audit Trail (추론 과정)

[**Dockerfile** 스니펫에서 ``USER`` 지시어 부재 확인] → [**ENTRYPOINT** 실행 시 기본 권한이 `root`일 가능성 판단] → [`root` 실행 상태는 애플리케이션 취약점 발생 시 영향도 확대] → [최소 권한 원칙 위반으로 **ISMS-P 2.5/2.6** 및 제로트러스트 정책과 불일치] → [`non-root` 사용자 명시 및 권한 축소로 수정 필요]

2. Docker 컨테이너가 non-root USER 없이 실행되어 root 권한으로 동작할 수 있음

| **Status:** TP | **Rule:** dockerfile.security.missing-user-entrypoint.missing-user-entrypoint | **File:** mini_project5\Dockerfile

최종 런타임 stage의 Dockerfile에 USER 지시어가 없고 ENTRYPOINT만 정의되어 있어, 컨테이너 프로세스가 기본적으로 root 권한으로 실행될 가능성이 높습니다. 애플리케이션 취약점이 존재할 경우 컨테이너 내부 권한 상승 및 호스트/볼륨/클러스터 영향 범위 확대 위험이 커집니다.

공격 시나리오: 공격자가 애플리케이션의 별도 취약점(예: 파일 업로드, 명령 실행, 템플릿 인젝션)을 통해 프로세스 실행 권한을 획득하면, 컨테이너가 root로 동작하는 상태에서 시스템 파일 접근, 민감 설정 읽기, writable mount 변조, 클러스터 내 lateral movement 시도가 가능해집니다.

취약 코드:

```
FROM amazoncorretto:17-alpine-jre
WORKDIR /app
COPY --from=builder /app/build/libs/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

수정 코드:

```
FROM amazoncorretto:17-alpine-jre
WORKDIR /app
COPY --from=builder /app/build/libs/*.jar app.jar
RUN addgroup -S app && adduser -S app -G app && chown -R app:app /app
USER app
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

수정 가이드: 최종 런타임 stage에 non-root 전용 계정을 생성하고 소유권을 부여한 뒤 USER를 해당 계정으로 전환하세요. 권한 전환 후에도 애플리케이션이 정상 동작하도록 /app 및 app.jar의 읽기 권한을 보장해야 합니다.

ISMS-P: ISMS-P 2.2.6 보안 위반 시 조치: 컨테이너 실행 권한을 최소권한으로 제한하지 않아 내부 정책 위반 및 보안사고 발생 시 통제 조치가 미흡할 수 있습니다. OT 제로트러스트 관점에서도 최소권한(least privilege) 원칙과 보호대상 최소화 원칙에 반합니다.

AI Audit Trail (추론 과정)

[최종 **Dockerfile stage**에서 실행 주체를 확인함] → [**USER** 지시어가 없어 **root** 실행 가능성을 확인함] → [**root** 실행은 애플리케이션 취약점 발생 시 권한 남용/컨테이너 장악으로 이어질 수 있음을 판단함] → [최소 권한 원칙 위반으로 **ISMS-P 2.2.6** 및 **OT 제로트러스트** 관점의 통제 미흡으로 매핑함]

3. Kubernetes 컨테이너 권한 상승 방지 설정 미흡

| **Status:** FP | **Rule:** yaml.kubernetes.security.allow-privilege-escalation-no-securitycontext.allow-privilege-escalation-no-securitycontext | **File:** mini_project5\cluster-autoscaler-autodiscover.yaml

Deployment의 컨테이너 보안 컨텍스트에 allowPrivilegeEscalation: false가 명시되지 않아 Semgrep 경고가 발생했습니다. 다만 본 스니펫에서는 클러스터 오토스케일러 공식 이미지 사용, 비루트 실행 설정, 그리고 권한상승으로 이어지는 구체적 공격 경로가 확인되지 않아 오탐으로 판단합니다.

오탐 사유: 제공된 스니펫 기준으로는 pod-level securityContext(runAsNonRoot/runAsUser/fsGroup)는 존재하지만, 컨테이너 레벨에서 allowPrivilegeEscalation

미설정만으로 실제 권한상승이 가능한 경로(사용자 제어 입력, 공격 가능한 `setuid/setgid` 바이너리, 추가 `capabilities`, `writable host mount` 등)가 확인되지 않았습니다. 따라서 현재 결과는 '보안 모범 사례 미준수' 경고에 가깝고, 스니펫만으로는 실제 취약점(TP)을 입증하기 부족합니다.

공격 시나리오: 현재 스니펫만으로는 구체적인 익스플로잇 시나리오를 구성하기 어렵습니다. 만약 이미지 내부에 `setuid/setgid` 바이너리나 추가 `capabilities`가 존재하고, 컨테이너가 쓰기 가능한 호스트 경로를 함께 마운트하고 있었다면 제한 우회 후 권한상승 시도가 가능할 수 있으나, 해당 증거는 제공되지 않았습니다.

수정 가이드: 컨테이너 레벨 `securityContext`에 `allowPrivilegeEscalation: false`를 명시하여 `setuid/setgid` 기반 권한상승을 차단하고, 가능하면 `capabilities` 전체 제거와 읽기 전용 루트 파일시스템을 추가해 컨테이너 탈취 시 영향 범위를 축소합니다.

ISMS-P: ISMS-P 관점에서는 안전한 설정 미흡이 운영상 보안통제 강화 필요사항에 해당할 수 있으나, 본 건은 개인정보 수집·이용(3.1.1)과 직접 관련된 취약점은 아닙니다. 다만 제로트러스트 원칙상 최소권한/기본 차단(OT 안내서의 보호 대상 최소화, 점진적 확장)과 부합하도록 컨테이너 권한을 최소화하는 것이 바람직합니다.

AI Audit Trail (추론 과정)

[Semgrep이 `allowPrivilegeEscalation` 미설정을 탐지함] → [스니펫에서 `pod-level securityContext`와 비루트 실행 설정을 확인함] → [사용자 제어 입력과 권한상승 싱크를 잇는 구체적 공격 경로는 확인되지 않음] → [클러스터 오토스케일러 공식 이미지 특성상 현재 조각만으로 TP 입증이 부족함] → [따라서 FP로 분류하고 컨테이너 `securityContext` 강화로 보완 권고]

4. Kubernetes 컨테이너 권한 상승 방지 설정 미흡

| **Status:** TP | **Rule:** `yaml.kubernetes.security.allow-privilege-escalation-no-securitycontext.allow-privilege-escalation-no-securitycontext` | **File:** `mini_project5\k8s-deployment.yaml`

backend 컨테이너에 `securityContext.allowPrivilegeEscalation: false`가 명시되어 있지 않아, 컨테이너 내부의 `setuid/setgid` 바이너리 또는 취약한 실행 흐름을 통해 권한 상승이 가능해질 수 있습니다. 이 설정은 컨테이너 프로세스가 불필요하게 상위 권한으로 상승하는 것을 차단하는 기본 방어선이므로, 누락 시 침해 시 피해 범위가 커질 수 있습니다.

공격 시나리오: 공격자가 애플리케이션 취약점(예: RCE, 파일 업로드, 서드파티 라이브러리 취약점)을 통해 backend 컨테이너 내부 셸 또는 프로세스 실행 권한을 획득한 뒤, setuid/setgid 바이너리나 로컬 권한 상승 기법을 활용해 컨테이너 내 권한을 확대할 수 있습니다. 그 결과 민감한 환경변수, 마운트된 볼륨, 서비스 계정 토큰, 네트워크 접근 범위에 더 넓게 접근할 가능성이 커집니다.

취약 코드:

```
containers:
  - name: backend
    image: 416225317325.dkr.ecr.us-east-1.amazonaws.com/ai0102-backend:
    ports:
      - containerPort: 8080
    resources:
      requests:
        cpu: "100m"
        memory: "256Mi"
      limits:
        cpu: "500m"
        memory: "512Mi"
    env:
      - name: SPRING_DATASOURCE_URL
        value: jdbc:postgresql://db-0.db.default.svc.cluster.local:5432
```

수정 코드:

```
containers:
  - name: backend
    image: 416225317325.dkr.ecr.us-east-1.amazonaws.com/ai0102-backend:
    securityContext:
      allowPrivilegeEscalation: false
    ports:
      - containerPort: 8080
    resources:
      requests:
        cpu: "100m"
        memory: "256Mi"
      limits:
        cpu: "500m"
        memory: "512Mi"
```

수정 가이드: backend 컨테이너에

`securityContext.allowPrivilegeEscalation: false`를 명시하여 프로세스 권한 상승을 차단해야 합니다. 가능하면 `runAsNonRoot: true`, `readOnlyRootFilesystem: true`, 불필요한 `capabilities` 제거까지 함께 적용하여 컨테이너 탈취 시에도 피해 확산을 최소화하세요.

ISMS-P: ISMS-P의 보안 구성 관리 및 최소권한 원칙에 부합하지 않습니다. 컨테이너 실행 시 권한 상승 방지 설정이 누락되면 침해 시 영향 범위가 커질 수 있으

며, OT/제로트러스트 관점에서도 보호 대상을 작게 정의하고(최소권한) 반복적으로 확장한다는 원칙에 반합니다.

AI Audit Trail (추론 과정)

[backend 컨테이너 정의를 확인했고 `securityContext`가 보이지 않음] → [Semgrep 룰이 요구하는 `allowPrivilegeEscalation: false` 미설정 상태로 판단] → [Kubernetes 기본 동작상 권한 상승 완화가 보장되지 않아 공격 표면 확대 가능] → [컨테이너 침해 후 setuid/setgid 또는 로컬 권한 상승 기법 악용 가능성 도출] → [ISMS-P 최소권한/보안구성 관리 및 제로트러스트 원칙 위반으로 매핑]

5. Kubernetes 컨테이너에 privilege escalation 방지 보안컨텍스트 미설정

| **Status:** TP | **Rule:** yaml.kubernetes.security.allow-privilege-escalation-no-securitycontext.allow-privilege-escalation-no-securitycontext | **File:** mini_project5\k8s-deployment.yaml

k8s-deployment.yaml의 postgres 컨테이너 정의에

securityContext.allowPrivilegeEscalation: false가 명시되지 않았 습니다. Semgrep 경고는 컨테이너가 setuid/setgid 바이너리 또는 권한상승 경로를 통해 불필요한 권한 획득으로 이어질 수 있음을 지적하며, Kubernetes Pod/컨테이너 수준에서 권한상승을 차단하도록 요구합니다. 이는 컨테이너 하드닝 부재로 인해 공격자가 컨테이너 내부에서 권한상승 후 민감 자원 접근 을 시도할 수 있는 상태입니다.

공격 시나리오: 공격자가 애플리케이션의 별도 취약점(RCE, 파일 업로드 악용, 의존성 취약점 등)을 통해 postgres 컨테이너 내부 실행 권한을 획득한 뒤, allowPrivilegeEscalation이 비활성화되지 않은 환경에서 setuid/setgid 바이너리 또는 잘못된 권한 설정을 악용하여 권한을 상승시킬 수 있습니다. 권한 상승 후 컨테이너 내 데이터 파일, 환경변수, 마운트된 시크릿, 내부 네트워크 접근 을 통해 민감정보 탈취나 추가 침투를 시도할 수 있습니다.

취약 코드:

```
spec:
  containers:
    - name: postgres
      image: postgres:15-alpine
      ports:
```

```

    - containerPort: 5432
  env:
    - name: POSTGRES_DB
      value: bookdb
    - name: POSTGRES_USER
      valueFrom:
        secretKeyRef:
          name: db-secret
          key: username
    - name: POSTGRES_PASSWORD
... (8줄 생략)

```

수정 코드:

```

spec:
  containers:
    - name: postgres
      image: postgres:15-alpine
      securityContext:
        allowPrivilegeEscalation: false
        runAsNonRoot: true
        capabilities:
          drop: ["ALL"]
      ports:
        - containerPort: 5432
      env:
        - name: POSTGRES_DB
          value: bookdb
        - name: POSTGRES_USER
... (13줄 생략)

```

수정 가이드: 해당 컨테이너에 `securityContext`를 명시하고 `allowPrivilegeEscalation: false`를 설정하여 권한상승을 차단해야 합니다. 추가로 `runAsNonRoot`, `capabilities.drop: ["ALL"]`를 적용하면 컨테이너 하드닝 수준이 향상됩니다. PostgreSQL 데이터 디렉터리는 볼륨으로 유지하되, 루트 권한이나 불필요한 Linux capability에 의존하지 않도록 배포 설정을 조정하세요.

ISMS-P: 주요정보통신기반시설 기술적 취약점 분석·평가 방법 상세가이드의 취지상, 컨테이너/서버 하드닝 미흡은 최소권한 및 권한상승 억제 통제를 충족하지 못하는 상태입니다. OT 제로트러스트 관점에서도 신뢰 경계 내부 컨테이너에 대해 권한상승을 기본 차단해야 하며, `allowPrivilegeEscalation` 미설정은 권한 상승 억제 통제 부재로 볼 수 있습니다.

AI Audit Trail (추론 과정)

[문제 위치가 **Kubernetes StatefulSet**의 **postgres** 컨테이너임을 확인] → [컨테이너 정의에서

`securityContext.allowPrivilegeEscalation` 미설정 확인] →
[Semgrep 룰이 권한상승 차단 하드닝 누락을 탐지하는 유형임을 식별] →
[컨테이너 침해 후 `setuid/setgid` 기반 권한상승 가능성 평가] →
[ISMS-P 및 OT 제로트러스트의 최소권한/권한상승 억제 통제 미충족으로 매핑]

6. Kubernetes 컨테이너 권한 상승 방지 설정 누락 여부 검증 필요

| **Status:** FP | **Rule:** `yaml.kubernetes.security.allow-privilege-escalation-no-securitycontext.allow-privilege-escalation-no-securitycontext` | **File:** `mini_project5\k8s-minikube.yaml`

Semgrep는 postgres 컨테이너 정의 위치에서 `allowPrivilegeEscalation=false`가 설정되지 않은 경우를 경고합니다. 다만 현재 제공된 자료만으로는 컨테이너/Pod 레벨 `securityContext`가 다른 위치에 존재하는지, 또는 다른 정책으로 보완되는지 확인되지 않아 오탐으로 단정할 수 없습니다. 원본 YAML 전체를 확인하면 실제로는 TP일 가능성도 있습니다.

오탐 사유: 제공된 스니펫만으로는 해당 컨테이너에 `securityContext`가 실제로 누락되었다는 사실을 최종 검증할 수 없었고, `read_source`로 원본 YAML을 열 수 없어 배포 스펙 전체 문맥을 확인하지 못했습니다. 또한 이 규칙은 '사용자 입력 →싱크'형 취약점이 아니라 Kubernetes 설정 누락 여부를 보는 구성 취약점이며, 현재 제공된 조각만으로는 `allowPrivilegeEscalation` 값의 존재/상속 여부를 확정할 수 없습니다.

공격 시나리오: 만약 postgres 컨테이너가 `allowPrivilegeEscalation=true`(기본값) 상태로 실행되고 `setuid/setgid` 바이너리 또는 추가 Linux capability가 존재한다면, 컨테이너 내부 침해 이후 권한 상승이 가능해질 수 있습니다. 공격자는 취약한 애플리케이션 또는 이미지 내 로컬 취약점을 악용해 더 높은 권한으로 파일/프로세스 접근 범위를 넓힐 수 있습니다.

수정 가이드: postgres 컨테이너에 `securityContext`를 명시하여 권한 상승을 차단해야 합니다. `allowPrivilegeEscalation=false`를 설정하고, 가능한 경우 `runAsNonRoot`와 `capabilities.drop: [ALL]`, `seccompProfile: RuntimeDefault`까지 함께 적용해 컨테이너 탈출 및 로컬 권한 상승 위험을 낮추는 것이 권장됩니다.

ISMS-P: ISMS-P 기술적 취약점 관점에서 컨테이너 실행 계정 및 권한 제한 미비는 최소권한 원칙 위반에 해당할 수 있습니다. 다만 본 건은 세션 타임아웃(C-14)이나 웹 세션 고정/인증우회(XS, IDOR) 같은 참조 규약과는 직접 대응하지 않으며, 제로트러스트 관점의 '항상 검증, 최소권한, 명시적 허용' 원칙에 반하는 배포

설정 문제로 해석하는 것이 적절합니다.

AI Audit Trail (추론 과정)

[Semgrep가 postgres 컨테이너의 privilege escalation 방지 설정 미흡 가능성을 탐지함] → [그러나 현재 세션에서 원본 k8s-minikube.yaml 전체 문맥을 read_source로 검증하지 못함] → [securityContext 존재 여부와 allowPrivilegeEscalation 값의 실제 설정을 확인할 수 없음] → [데이터 흐름 취약점이 아닌 Kubernetes 구성 점검 항목이므로 문맥 부족 시 TP 단정 불가] → [따라서 본 건은 현 단계에서 오탐 보류 (검증 불가)로 처리하고 추가 원본 확인이 필요함]

7. Kubernetes 컨테이너 권한 상승 방지 설정 누락

| **Status:** TP | **Rule:** yaml.kubernetes.security.allow-privilege-escalation-no-securitycontext.allow-privilege-escalation-no-securitycontext | **File:** mini_project5\k8s-deployment.yaml

frontend 컨테이너에 securityContext가 정의되어 있지 않아 allowPrivilegeEscalation: false로 권한 상승을 차단하는 방어가 누락되었습니다. 이 설정이 없으면 컨테이너 내부에서 setuid/setgid 바이너리 또는 다른 로컬 권한상승 경로가 존재할 경우, 침해 후 컨테이너 권한이 상승될 수 있습니다. YAML 문맥상 명시적인 차단 정책이 없으므로 오탐으로 보기 어렵고 실제 보안 설정 미비로 판단됩니다.

공격 시나리오: 공격자가 프론트엔드 애플리케이션의 다른 취약점(RCE, 파일 업로드 악용 등)으로 컨테이너 내부 셸을 확보한 뒤, setuid/setgid 바이너리나 로컬 권한상승 경로를 이용해 root 권한 또는 더 높은 컨테이너 권한을 획득할 수 있습니다. 이후 마운트된 볼륨, 서비스 계정 토큰, 내부 네트워크 자원에 대한 접근 범위가 확대될 수 있습니다.

취약 코드:

```
containers:
  - name: frontend
    image: 416225317325.dkr.ecr.us-east-1.amazonaws.com/ai01002-frontend:latest
    ports:
      - containerPort: 3000
    resources:
      requests:
        cpu: "100m"
        memory: "256Mi"
```

```
limits:
  cpu: "500m"
  memory: "512Mi"
```

수정 코드:

```
containers:
  - name: frontend
    image: 416225317325.dkr.ecr.us-east-1.amazonaws.com/ai0102-front:la
    securityContext:
      allowPrivilegeEscalation: false
      runAsNonRoot: true
      capabilities:
        drop: ["ALL"]
    ports:
      - containerPort: 3000
    resources:
      requests:
        cpu: "100m"
        memory: "256Mi"
      limits:
... (2줄 생략)
```

수정 가이드: 각 컨테이너에 securityContext를 명시하고 allowPrivilegeEscalation을 false로 설정해 권한 상승을 차단해야 합니다. 가능하다면 runAsNonRoot, capabilities.drop: ["ALL"]도 함께 적용하여 최소권한 원칙을 강화하세요. 단, 애플리케이션 실행에 영향이 있을 수 있으므로 이미지 동작 검증 후 배포해야 합니다.

ISMS-P: ISMS-P 2.6 접근통제 및 2.10 시스템 및 서비스 보안관리 관점에서 컨테이너 권한 상승 방지 기준이 미흡합니다. 또한 주요정보통신기반시설 기술적 취약점 분석·평가 방법 상세가이드의 보안 설정 미비 항목 취지와도 부합하며, 최소권한/명시적 차단 원칙을 충족하지 못합니다.

AI Audit Trail (추론 과정)

[Deployment의 frontend 컨테이너 정의를 확인] →
[securityContext 및 allowPrivilegeEscalation 설정 부재를 확인]
→ [권한 상승 차단 로직이 없으므로 로컬 privilege escalation 위험을 도출] → [공격 후 컨테이너 내 setuid/setgid 또는 유사 경로 악용 가능성을 고려] → [ISMS-P 2.6 접근통제 및 2.10 보안관리 관점의 설정 미비로 매핑]

8. Kubernetes 컨테이너 권한 상승 방지 설정 미흡

| **Status:** TP | **Rule:** `yaml.kubernetes.security.allow-privilege-escalation-no-securitycontext.allow-privilege-escalation-no-securitycontext` | **File:** `mini_project5\k8s-deployment.yaml`

nginx 컨테이너 정의에 `securityContext`가 보이지 않으며, `allowPrivilegeEscalation: false`가 명시되지 않았습니다. Kubernetes에서는 컨테이너 이미지 내부의 `setuid/setgid` 바이너리 또는 잘못된 권한 설정을 통해 프로세스 권한 상승이 발생할 수 있으므로, 컨테이너 단위로 권한 상승을 금지해야 합니다. 현재 구성은 해당 방어가 누락되어 있어 런타임 권한 상승 가능성을 낮추지 못합니다.

공격 시나리오: 공격자가 컨테이너 내부에서 명령 실행 권한을 획득한 뒤, 이미지 내 `setuid/setgid` 바이너리 또는 권한이 과도한 실행 파일을 이용해 권한을 상승시킬 수 있습니다. 권한 상승 후에는 컨테이너 내 민감 설정, 마운트된 볼륨, 환경변수, 네트워크 자격정보 등을 추가로 탈취하거나, 동일 노드의 다른 자원에 대한 공격 발판을 확보할 수 있습니다.

취약 코드:

```
containers:
  - name: nginx
    image: nginx:alpine
    ports:
      - containerPort: 80
    readinessProbe:
      tcpSocket:
        port: 80
      initialDelaySeconds: 10
      periodSeconds: 5
    resources:
      requests:
        cpu: "100m"
        memory: "256Mi"
      limits:
    ... (6줄 생략)
```

수정 코드:

```
containers:
  - name: nginx
    image: nginx:alpine
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      runAsNonRoot: true
```

```
capabilities:
  drop:
    - ALL
ports:
  - containerPort: 80
readinessProbe:
  tcpSocket:
    port: 80
... (13줄 생략)
```

수정 가이드: 컨테이너 단위 `securityContext`를 추가하고 `allowPrivilegeEscalation: false`를 명시하여 프로세스 권한 상승을 차단합니다. 운영 안정성과 추가 하드닝을 위해 `runAsNonRoot`, `readOnlyRootFilesystem`, `capabilities.drop: [ALL]`도 함께 적용하는 것을 권장합니다.

ISMS-P: ISMS-P 기술적 취약점 관리 관점에서 컨테이너 권한 최소화 및 접근통제 설정이 미흡합니다. 주요정보통신기반시설 기술적 취약점 분석·평가 방법 상세가이드의 WEB-17(불필요한 가상 디렉터리 삭제) 취지처럼 공격 가능 영역을 최소화해야 하며, 권한 상승을 허용하는 배포 설정은 제로트러스트 원칙(기본 불신, 최소권한)에 반합니다.

AI Audit Trail (추론 과정)

[Deployment의 `nginx` 컨테이너 정의를 확인] →
 [`securityContext` 및 `allowPrivilegeEscalation` 설정이 보이지 않음] → [setuid/setgid 기반 런타임 권한 상승 가능성을 차단하지 못함] → [컨테이너 권한 최소화 미준수로 판단] → [ISMS-P 최소권한·공격면 축소 요구와 부합하지 않아 취약점으로 매핑]

9. Kubernetes 컨테이너 권한 상승 방지 설정 미흡

| **Status:** TP | **Rule:** yaml.kubernetes.security.allow-privilege-escalation-no-securitycontext.allow-privilege-escalation-no-securitycontext | **File:** mini_project5\k8s-minikube.yaml

Deployment의 backend 컨테이너에

`securityContext.allowPrivilegeEscalation: false`가 명시되지 않아, 컨테이너 내부에 `setuid/setgid` 바이너리 또는 다른 권한 상승 경로가 존재할 경우 프로세스 권한 상승을 제한하지 못합니다. 제공된 스니펫에는 상위 Pod 수준의 보안 컨텍스트 강제 설정도 확인되지 않아 Semgrep 경고는 타당합니다.

다.

공격 시나리오: 공격자가 백엔드 컨테이너 내부에서 코드 실행 권한을 획득한 뒤, 이미지 내 `setuid/setgid` 바이너리나 허술한 런타임 기능을 이용해 루트 권한으로 상승을 시도합니다. 이후 컨테이너 내 비밀값, 마운트된 볼륨, 네트워크 접근 권한을 악용해 DB 자격증명 또는 내부 서비스로 확장 침투할 수 있습니다.

취약 코드:

```
containers:
  - name: backend
    image: backend:latest
    imagePullPolicy: Never
    ports:
      - containerPort: 8080
```

수정 코드:

```
containers:
  - name: backend
    image: backend:latest
    imagePullPolicy: Never
    securityContext:
      allowPrivilegeEscalation: false
      runAsNonRoot: true
      readOnlyRootFilesystem: true
      capabilities:
        drop: ["ALL"]
    ports:
      - containerPort: 8080
```

수정 가이드: backend 컨테이너에 `securityContext`를 추가하고 `allowPrivilegeEscalation: false`를 명시합니다. 가능하다면 비루트 실행, 읽기 전용 루트 파일시스템, 모든 capability 제거를 함께 적용해 컨테이너 탈출 및 권한 상승의 영향을 최소화합니다. `initContainer`에도 동일한 보안 원칙을 검토해야 합니다.

ISMS-P: ISMS-P 2.10.2 클라우드 보안 및 2.11.2 취약점 점검 및 조치 관점에서 컨테이너 실행 보안 통제가 미흡합니다. 또한 권한 상승 방지 미설정은 최소권한 원칙(OT 제로트러스트 관점의 자원별 최소 권한 부여) 위반으로 해석될 수 있으며, 운영 배포 구성의 보안 요구사항 정의·검토 미흡(2.8.1, 2.8.2)에도 해당합니다.

AI Audit Trail (추론 과정)

[컨테이너 배포 **YAML**의 **backend** 스펙 확인] → [``securityContext`` 및 ``allowPrivilegeEscalation`` 설정 부재 확인] → [기본 권한 상승 차단이 보장되지 않아 **privilege escalation** 위험 도출] → [공격 시

나리오: 컨테이너 내 코드 실행 후 **setuid/setgid** 등으로 권한 상승 가능] → [ISMS-P 2.10.2/2.11.2 및 최소권한 원칙 관점에서 개선 필요]

10. Kubernetes 컨테이너 권한 상승 방지 설정 미흡

| **Status:** TP | **Rule:** yaml.kubernetes.security.allow-privilege-escalation-no-securitycontext.allow-privilege-escalation-no-securitycontext | **File:** mini_project5\k8s-minikube.yaml

frontend Deployment의 컨테이너 정의에서 securityContext가 보이지 않으며, 특히 allowPrivilegeEscalation: false가 설정되지 않았습니다. Kubernetes 컨테이너는 기본적으로 setuid/setgid 바이너리 등으로 권한 상승 가능성이 존재할 수 있으므로, 해당 옵션을 명시적으로 비활성화하지 않으면 컨테이너 내부 프로세스의 불필요한 권한 상승을 허용할 수 있습니다. 이는 컨테이너 격리와 최소권한 원칙에 반하며, 이미지가 침해되었을 때 호스트/클러스터 내 민감 자원 접근 범위를 확대할 수 있습니다.

공격 시나리오: 공격자가 애플리케이션 RCE나 이미지 취약점을 통해 컨테이너 내부에 진입한 뒤, setuid/setgid 바이너리 또는 과도한 실행 권한을 악용해 더 높은 권한으로 승격할 수 있습니다. 이후 컨테이너 내부 자격증명, 마운트된 볼륨, 환경변수, 네트워크 접근 권한 등을 이용해 추가 침투나 정보 탈취로 이어질 수 있습니다.

취약 코드:

```
- name: frontend
  image: frontend:latest
  imagePullPolicy: ifNotPresent
  ports:
    - containerPort: 3000
  resources:
    requests:
      cpu: "100m"
      memory: "256Mi"
    limits:
      cpu: "500m"
      memory: "512Mi"
```

수정 코드:

```
- name: frontend
  image: frontend:latest
  imagePullPolicy: ifNotPresent
```

```

securityContext:
  allowPrivilegeEscalation: false
  runAsNonRoot: true
  readOnlyRootFilesystem: true
ports:
  - containerPort: 3000
resources:
  requests:
    cpu: "100m"
    memory: "256Mi"
  limits:
    cpu: "500m"
... (1줄 생략)

```

수정 가이드: 해당 컨테이너에 securityContext를 추가하고 allowPrivilegeEscalation을 false로 지정하여 setuid/setgid 기반 권한 상승을 차단해야 합니다. 가능하면 runAsNonRoot, readOnlyRootFilesystem, capabilities drop 설정까지 함께 적용하여 최소권한 실행 원칙을 강화하는 것이 좋습니다.

ISMS-P: ISMS-P 관점에서 컨테이너 실행 권한을 최소화하지 않아 기술적 취약점 관리가 미흡합니다. 특히 내부 보안정책 위반 시 조치가 요구되는 2.2.6 보안 위반 시 조치 기준과 연결되며, OT/제로트러스트 관점의 최소권한 원칙에도 반합니다. 참조 가이드의 U-11(불필요한 shell 제거), HV-18(비인가 설정 변경 방지)과 같이 불필요한 권한/변경 가능성을 차단해야 하는 통제 취지와도 유사합니다.

AI Audit Trail (추론 과정)

[frontend Deployment 컨테이너 블록에서 securityContext 부재 확인] → [allowPrivilegeEscalation 차단 설정이 없어 권한 상승 방어 미적용으로 판단] → [컨테이너 내부 setuid/setgid 또는 과도한 권한 악용 가능성 도출] → [최소권한 원칙 위반 및 Kubernetes 권한 상승 취약점으로 결론] → [ISMS-P 2.2.6 및 U-11/HV-18 취지에 따라 보안 통제 강화 필요]

11. Kubernetes nginx 컨테이너의 privilege escalation 차단 설정 누락

| **Status:** TP | **Rule:** yaml.kubernetes.security.allow-privilege-escalation-no-securitycontext.allow-privilege-escalation-no-securitycontext | **File:** mini_project5\k8s-minikube.yaml

k8s-minikube.yaml의 nginx Deployment 컨테이너 정의에 securityContext가 존재하지 않아 allowPrivilegeEscalation 차단이 적용되지 않았습니다. Kubernetes 기본 동작상 컨테이너는 setuid/setgid 바이너리나 런타임 취약점이 존재할 경우 권한 상승 위험이 남아 있으며, 명시적으로 allowPrivilegeEscalation: false를 설정해야 합니다. 본 건은 입력값 기반의 오탐이 아니라, 배포 설정 누락으로 인한 실제 보안 구성 취약점입니다.

공격 시나리오: 공격자가 nginx 컨테이너 내부에서 취약한 바이너리(setuid/setgid 포함)나 별도 취약점을 악용할 수 있는 상황에서, privilege escalation 제한이 없으면 컨테이너 내 더 높은 권한으로 전환될 가능성이 있습니다. 그 결과 컨테이너 탈출 전 단계의 권한 확대, 민감 파일 접근, 후속 공격 준비가 쉬워집니다.

취약 코드:

```
containers:
  - name: nginx
    image: nginx:alpine
    ports:
      - containerPort: 80
    readinessProbe:
      tcpSocket:
        port: 80
    resources:
      requests:
        cpu: "100m"
        memory: "256Mi"
      limits:
        cpu: "500m"
        memory: "512Mi"
```

수정 코드:

```
containers:
  - name: nginx
    image: nginx:alpine
    securityContext:
      allowPrivilegeEscalation: false
      runAsNonRoot: true
      readOnlyRootFilesystem: true
      capabilities:
        drop:
          - ALL
    ports:
      - containerPort: 80
    readinessProbe:
      tcpSocket:
        port: 80
    ... (9줄 생략)
```

수정 가이드: nginx 컨테이너에 securityContext를 추가하고

`allowPrivilegeEscalation`을 `false`로 명시하세요. 가능하면 `runAsNonRoot`, `readOnlyRootFilesystem`, `capabilities.drop: ALL`도 함께 적용해 컨테이너 권한을 최소화해야 합니다. 이는 ISMS-P의 최소권한 및 안전한 시스템 설정 원칙에 부합하며, 제로트러스트 관점에서도 기본 신뢰를 배제하고 실행 권한을 엄격히 제한하는 조치입니다.

ISMS-P: ISMS-P 기술적 취약점 관점에서 안전하지 않은 컨테이너 실행 설정이며, 최소권한 원칙 위반에 해당합니다. 제공된 가이드의 취지상 웹/플랫폼 보안 설정은 명시적 차단과 안전한 기본값 적용이 요구되며, 본 건은 컨테이너 권한 상승 방지 설정이 누락된 상태입니다.

AI Audit Trail (추론 과정)

[컨테이너 보안 설정 누락 여부 확인 대상 식별] → [nginx 컨테이너에 `securityContext` 부재 확인] → [`allowPrivilegeEscalation` 차단 미적용으로 권한 상승 가능성 도출] → [Kubernetes 배포 보안 설정 결함으로 **True Positive** 판정] → [`allowPrivilegeEscalation:false` 및 최소권한 옵션 추가 권고]

12. HTTP 메서드 미지정 RequestMapping(화면 조회용)

| **Status:** FP | **Rule:** java.spring.security.unrestricted-request-mapping.unrestricted-request-mapping | **File:** mini_project5\book\src\main\java\com\aivle0102\book\controller\UserContrc

해당 메서드는 `/login/form` 로그인 화면을 반환하거나 세션 존재 시 `/main`으로 리다이렉트하는 단순 조회용 엔드포인트입니다. 입력값에 대한 조작이나 세션/DB 상태 변경이 없으므로, 본 경고는 현재 코드 문맥에서는 실제 CSRF 취약점으로 연결되지 않습니다. 다만 의도 명확화를 위해 GET 메서드로 제한하는 것이 바람직합니다.

13. 하드코딩된 더미 JWT 문자열로 인한 비밀정보 탐지

| **Status:** FP | **Rule:** generic.secrets.security.detected-jwt-token.detected-jwt-token | **File:** mini_project5\book\frontend\src\app\userpage\view\page.js

프론트엔드 소스에 JWT처럼 보이는 Bearer 토큰 문자열이 정적으로 포함되어 있어 시크릿 스캐너가 탐지한 사례입니다. 그러나 해당 값은 FAKE_ACCESS_TOKEN으로 명명되어 있고, 로그인 구현 전 임시값이라는 주석이 존재하므로 운영 비밀 토큰 노출로 보기 어렵습니다. 다만 실제 서비스 배포 시에는 더미 JWT를 제거하고 환경변수/서버 발급 토큰으로 대체해야 합니다.

14. Kubernetes 컨테이너 비루트 실행 강제 설정 누락

| **Status:** TP | **Rule:** yaml.kubernetes.security.run-as-non-root.run-as-non-root | **File:** mini_project5\k8s-deployment.yaml

backend Deployment의 pod spec에서 컨테이너/파드 보안 컨텍스트가 확인되지 않아 `runAsNonRoot: true`가 강제되지 않습니다. 이미지가 root로 실행될 수 있고, 애플리케이션 침해 시 컨테이너 내부 권한상승 및 호스트 자원 접근 위험이 증가합니다. 공격자가 직접 입력을 제어하는 유형은 아니지만, 기본 하드닝 누락으로 인해 침해 후 피해 범위가 커지는 구성 취약점입니다.

15. Kubernetes 컨테이너 비루트 실행 설정 (runAsNonRoot) 누락 권고

| **Status:** FP | **Rule:** yaml.kubernetes.security.run-as-non-root.run-as-non-root | **File:** mini_project5\k8s-deployment.yaml

Kubernetes Deployment의 frontend 컨테이너에 `securityContext.runAsNonRoot: true` 설정이 확인되지 않는 것으로 보이며, Semgrep는 컨테이너가 root로 실행될 가능성을 경고합니다. 다만 현재 제공된 문맥만으로는 실제 root 실행 여부, 상위 PodSecurityContext 상속 여부, Docker 이미지 내부 USER 설정 여부를 확정할 수 없어 보안 위반을 단정하기 어렵습니다. 이 항목은 실제 악용 가능한 입력 기반 취약점보다는 운영 보안 강화를 위한 구성 권고에 가깝습니다.

16. Kubernetes Pod/컨테이너의 비루트 실행 보장 미설정

| **Status:** TP | **Rule:** yaml.kubernetes.security.run-as-non-root.run-as-non-root | **File:** mini_project5\k8s-deployment.yaml

StatefulSet의 postgres 컨테이너 정의에서

`securityContext.runAsNonRoot: true`가 보이지 않아, 컨테이너가 root로 실행될 가능성을 배제할 수 없습니다. 이는 컨테이너 탈출/권한상승/호스트 리소스 접근 위험을 키우며, 최소권한 원칙과 컨테이너 실행권한 제한 관점에서 보안 설정 미흡입니다. 제공된 스니펫 기준으로는 상위 템플릿이나 다른 매니페스트에서 보완된 흔적도 확인되지 않았습니다.

17. Kubernetes 컨테이너 비루트 실행 보장 설정 누락

| **Status:** FP | **Rule:** yaml.kubernetes.security.run-as-non-root.run-as-non-root | **File:** mini_project5\k8s-deployment.yaml

nginx Deployment의 Pod spec에서 컨테이너 및 initContainer에

`runAsNonRoot: true`가 명시되지 않아, 컨테이너가 root로 실행될 가능성을 배제할 수 없습니다. 다만 현재 스니펫만으로는 이미지 기본 사용자(root/non-root) 또는 별도 Pod/Container securityContext 존재 여부를 확인할 수 없어, 실제 취약점 확보정보는 보안 강화 권고 수준입니다.

18. Kubernetes Pod에 비루트 실행 보안 설정 누락 경고

| **Status:** FP | **Rule:** yaml.kubernetes.security.run-as-non-root.run-as-non-root | **File:** mini_project5\k8s-minikube.yaml

Kubernetes StatefulSet의 db Pod에서 컨테이너가 비루트로 실행되도록 강제하는 `securityContext.runAsNonRoot: true` 설정이 제공 스니펫에서 확인되지 않습니다. 다만 이 조치는 공격자 입력이 실행 경로로 이어지는 취약점이 아니라, 컨테이너 권한 상승 위험을 낮추기 위한 하드닝 권고입니다. 현재 문맥만으로는 실제 root 실행 여부를 증명할 수 없어 보안 설정 누락 경고로 판단됩니다.

19. Kubernetes 백엔드 컨테이너가 비루트 실행 보장을 명시하지 않음

| **Status:** TP | **Rule:** yaml.kubernetes.security.run-as-non-root.run-as-non-root | **File:** mini_project5\k8s-minikube.yaml

제공된 스니펫 기준으로 backend Deployment의 pod spec(146행 부근)에 `securityContext.runAsNonRoot: true`가 확인되지 않습니다. Kubernetes 컨테이너가 root 권한으로 실행될 경우 컨테이너 탈출, 권한 상승, 노드 내 민감 자원 접근 위험이 커집니다. 이 탐지는 우선순위는 낮더라도 실제 보안 통제 누락으로 보는 것이 타당합니다.

20. Kubernetes 컨테이너 non-root 실행 설정 누락 여부 검토 필요

| **Status:** FP | **Rule:** yaml.kubernetes.security.run-as-non-root.run-as-non-root | **File:** mini_project5\k8s-minikube.yaml

이 탐지는 컨테이너가 root로 실행될 가능성을 경고하며 `securityContext.runAsNonRoot: true` 설정을 권장합니다. 그러나 현재 확보된 코드 조각만으로는 해당 Deployment에 이미 다른 보안 컨텍스트가 적용되어 있는지, 혹은 이미지가 non-root 사용자로 동작하는지 확인되지 않아 실제 취약점으로 확정할 수 없습니다.

21. Kubernetes 컨테이너에 비루트 실행 보안 설정 미흡

| **Status:** TP | **Rule:** yaml.kubernetes.security.run-as-non-root.run-as-non-root | **File:** mini_project5\k8s-minikube.yaml

Deployment의 Pod template spec 아래 `initContainer` 및 `container`에 `securityContext.runAsNonRoot: true`가 확인되지 않아, 컨테이너가 root로 실행될 가능성이 남아 있습니다. 제공된 스니펫 기준으로는 루트 권한 실행을 차단하는 방어 로직이 없으므로, 권한 상승 및 컨테이너 탈출 피해를 줄이기 위한 기본 하드닝이 누락된 상태입니다.

2.2 비즈니스 로직 심층 분석 결과

1. 클라이언트 저장 **userId**를 신뢰하는 비밀번호 변경 요청으로 인한 권한 우회(IDOR) 가능성

| **Type:** IDOR | **File:**

mini_project5\book\frontend\src\app\password\change\page.js

비밀번호 변경 화면에서 로그인 식별자(userId)를 localStorage의 'user' 객체에서 읽어 요청 바디에 그대로 포함합니다. 이 구조는 클라이언트가 임의로 변경 가능한 식별자를 서버로 전달하는 형태이므로, 백엔드(/api/user/change-pw)가 인증 세션/JWT의 주체와 요청 바디의 userId 일치 여부를 서버 측에서 강제 검증하지 않으면 타 계정 비밀번호 변경으로 이어질 수 있습니다. 현재 제공된 프론트 코드만으로 서버 검증 존재를 확인할 수 없어, 실제 공격 가능성은 백엔드 구현에 의존하지만 비즈니스 로직 결함 후보로는 명확합니다.

공격 시나리오: 공격자가 브라우저 개발자도구나 프록시로 localStorage의 user.userId 값을 다른 사용자 ID로 바꾼 뒤 비밀번호 변경 요청을 전송한다. 서버가 요청 바디의 userId를 신뢰하고 현재 인증된 세션 사용자와의 일치 검증을 하지 않으면, 피해자 계정의 비밀번호가 변경되어 계정 탈취가 가능하다.

취약 코드:

```
const user = JSON.parse(stored);
setUserId(user.userId);
...
body: JSON.stringify({
  userId,
  currentPassword,
  newPassword,
}),
```

수정 코드:

```
// 서버 측에서 인증 주체를 기준으로 비밀번호 변경
@PostMapping('/api/user/change-pw')
public ResponseEntity<?> changePassword(@AuthenticationPrincipal Custom
                                     @RequestBody ChangePwRequest re
    Long authUserId = principal.getUserId();
    userService.changePassword(authUserId, req.getCurrentPassword(), re
    return ResponseEntity.ok(Map.of('status', 'success'));
}
```

수정 가이드: 요청 바디의 userId를 권한 판단 근거로 사용하지 말고, 세션/JWT 등 서버가 신뢰하는 인증 컨텍스트에서 사용자 식별자를 추출해야 합니다

다. 또한 비밀번호 변경 시에는 현재 비밀번호 검증, 계정 소유자 일치 검증, 실패 횟수 제한을 서버에서 일관되게 수행해야 합니다.

ISMS-P: ISMS-P 웹 취약점 분석 항목 18. 쿠키 변조/임의 사용자 권한 탈취 관점의 권한 검증 미흡 가능성. 또한 서버 사이드 세션 검증 없이 클라이언트가 전달한 식별자를 신뢰하는 구조는 주요정보통신기반시설 상세가이드의 타 사용자 권한 탈취 시도/검증 기준(서버 측 인가 체크) 취지에 위배될 수 있습니다.

AI Audit Trail (추론 과정)

[비밀번호 변경 요청에서 사용자 식별자 전달 방식 확인] →
[localStorage 기반 **userId**가 클라이언트에서 조작 가능함을 확인] →
[요청 바디 **userId**가 권한 결정 인자로 사용되어 **IDOR** 가능성 도출] →
[서버가 인증 주체와 요청 **userId**를 대조하지 않으면 타 계정 변경 가능]
→ [ISMS-P 서버 사이드 세션/권한 검증 요구사항과 매핑]

2. 작품 등록 API의 **userId**를 클라이언트가 직접 지정하는 권한 우회 위험

| **Type:** IDOR | **File:**

mini_project5\book\frontend\src\app\userpage\create\page.js

작품 등록 요청이 `/api/book/insertByUrl?userId=...` 형태로 전송되며, `userId` 값이 `localStorage`의 사용자 객체에서 그대로 유래합니다. 프론트엔드만으로는 서버 검증을 확정할 수 없지만, 백엔드가 인증 주체와 요청 파라미터의 `userId`를 강제 대조하지 않으면 공격자가 `userId`를 조작해 타 계정 명의로 작품을 등록하는 IDOR/권한 우회가 가능합니다. 이는 비즈니스 로직 상 인증·권한 경계가 클라이언트 입력에 의존하는 구조로, 서버 사이드 접근통제 부재 시 실제 악용 가능합니다.

공격 시나리오: 공격자는 브라우저 개발자도구에서 `localStorage.user` 또는 요청 URL의 `userId`를 임의의 값으로 변경한 뒤 작품 등록을 수행한다. 서버가 요청 파라미터를 그대로 신뢰하면 작품이 다른 사용자 계정에 귀속되거나 권한 없는 등록이 발생한다.

취약 코드:

```
const userData = JSON.parse(localStorage.getItem("user"));  
...  
const response = await fetch(`/api/book/insertByUrl?userId=${userData.u  
method: "POST",
```

```
headers: { "Content-Type": "application/json" },
body: JSON.stringify(bookData),
});
```

수정 코드:

```
const response = await fetch(`/api/book/insertByUrl`, {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "Authorization": `Bearer ${token}`
  },
  body: JSON.stringify(bookData),
});

// 서버 측에서는 인증된 주체만 사용
@PostMapping("/api/book/insertByUrl")
public ResponseEntity<?> insert(@AuthenticationPrincipal UserPrincipal
                                @RequestBody BookData bookData) {
    bookService.insert(principal.getUserId(), bookData);
    return ResponseEntity.ok().build();
    ... (1줄 생략)
```

수정 가이드: `userId`를 클라이언트 입력이나 로컬스토리지 값에 의존하지 말고, 서버에서 인증 토큰/세션의 주체를 기준으로 귀속 사용자를 결정해야 합니다. 또한 요청 파라미터로 전달된 `userId`는 무시하거나 서버에서 주체와 일치 여부를 엄격히 검증해야 합니다. ISMS-P 관점에서는 웹 애플리케이션 권한 검증 및 세션/토큰 기반 접근통제 미흡으로 볼 수 있으며, 참조 3의 권한 검증 로직 추가 및 서버 사이드 접근통제 원칙에 위배됩니다.

ISMS-P: 참조 3의 웹 애플리케이션 권한 검증 미흡(인증이 필요한 주요 정보/기능에 대해 요청자의 권한을 서버 사이드에서 검증해야 함) 위반. 클라이언트가 조작 가능한 `userId`를 신뢰하는 구조는 비인가자 접근 통제 실패로 해석 가능.

AI Audit Trail (추론 과정)

[작품 등록 흐름에서 사용자 식별자가 어디서 오는지 확인] →
 [‘localStorage’의 `userId`가 쿼리스트링으로 서버에 전달되는 점을 확인] →
 [클라이언트 제공 `userId`를 서버가 그대로 신뢰하면 계정 귀속 조작이 가능하다고 판단] →
 [ISMS-P 참조 3의 서버 사이드 권한 검증/접근통제 미흡과 매핑] →
 [따라서 IDOR/권한 우회 가능성을 HIGH로 정리]

3. 작품 삭제 API의 소유권 검증 부재로 인한 IDOR 가능

성

| **Type:** IDOR | **File:**

mini_project5\book\frontend\src\app\userpage\view\page.js

프론트엔드가 작품 삭제를 수행할 때 `/api/book/delete/${idToDelete}` 형태로 오직 작품 ID만 전달합니다. 인증 헤더는 사용하지 않고, 서버가 요청 사용자와 삭제 대상 작품의 소유권을 교차 검증하지 않으면 다른 사용자의 작품도 삭제 가능한 IDOR가 발생합니다. 현재 코드만으로 백엔드 검증 유무는 확인할 수 없지만, 비즈니스 로직상 취약 경로가 명확합니다.

공격 시나리오: 공격자가 브라우저 콘솔이나 프록시로 DELETE 요청의 `idToDelete` 값을 다른 사용자의 작품 ID로 변경해 직접 호출하면, 백엔드가 소유권 검증을 하지 않을 경우 타인의 작품을 삭제할 수 있습니다.

취약 코드:

```
const response = await fetch(`/api/book/delete/${idToDelete}`, { method
```

수정 코드:

```
const ownerId = req.user.id;
const book = await Book.findByPk(req.params.id);
if (!book || book.ownerId !== ownerId) {
  return res.status(403).json({ message: 'Forbidden' });
}
await book.destroy();
return res.json({ message: 'deleted' });
```

수정 가이드: 삭제 API는 클라이언트가 제공한 ID만 믿지 말고, 서버에서 인증된 사용자 식별자와 대상 리소스의 소유권을 반드시 비교해야 합니다. 불일치 시 403을 반환하고, 클라이언트는 Authorization 없이 직접 삭제를 수행하지 않도록 세션/JWT 기반 인증을 적용해야 합니다.

ISMS-P: ISMS-P 웹 취약점 중 권한 검증 미흡으로 인한 접근통제 위반에 해당합니다. 제로트러스트 기본 원리(모든 접근에 대해 명시적 신뢰 검증, 최소 권한 부여) 및 OT/중요 시스템의 세션 기반 접근통제 원칙과도 배치됩니다.

AI Audit Trail (추론 과정)

[삭제 기능의 요청 경로와 파라미터 구조 확인] → [작품 ID만으로 DELETE 호출하는 점 식별] → [서버 측 소유권 검증 부재 시 타인 자원 삭제 가능성 도출] → [제로트러스트의 최소 권한/명시적 검증 원칙과 ISMS-P 권한통제로 매핑]

적절합니다.

AI Audit Trail (추론 과정)

[수정 기능의 인증/식별자 전달 방식을 점검] → [클라이언트 `localStorage`의 `userId`를 신뢰 입력으로 사용함을 확인] → [URL의 리소스 ID와 `body`의 `userId` 조합이 IDOR 우회 경로가 될 수 있음을 판단] → [서버 인증 주체 기반 소유권 검증 필요성 및 ISMS-P 최소권한 원칙으로 매핑]

5. 비밀번호 변경 API에서 세션 사용자 검증 누락으로 계정 탈취성 권한우회 가능

| **Type:** IDOR | **File:**

`mini_project5\book\src\main\java\com\ai_vle0102\book\controller\AuthContrc`

`/user/change-pw`는 요청 바디의 `userId`를 그대로 비밀번호 변경 대상 식별자로 사용하지만, 현재 로그인한 세션의 사용자 ID와 일치하는지 검증하지 않습니다. 따라서 서비스 레이어에서 별도 강제 검증이 없다면, 공격자는 자신의 세션을 유지한 채 다른 사용자의 `userId`를 지정해 비밀번호 변경을 시도할 수 있어 IDOR/권한우회로 이어질 수 있습니다. 이는 인증된 사용자의 계정 권한을 이용해 타 계정의 비밀정보를 변경하는 비즈니스 로직 결함입니다.

공격 시나리오: 공격자는 정상 계정으로 로그인한 뒤 `/user/change-pw` 요청의 `userId`를 타 사용자 값으로 바꾸고, 현재 비밀번호를 우회 가능한 형태로 맞추거나 서비스의 검증 취약점을 이용해 타 계정 비밀번호 변경을 시도한다. 서비스 레이어가 입력된 `userId`만 신뢰한다면 공격자는 다른 사용자 계정을 탈취할 수 있다.

취약 코드:

```
userService.changePassword(  
    request.getUserId(),  
    request.getCurrentPassword(),  
    request.getNewPassword()  
);
```

수정 코드:

```
@PostMapping("/change-pw")  
public ResponseEntity<?> changePassword(  

```

```

        @Valid @RequestBody ChangePasswordRequest request,
        BindingResult bindingResult,
        HttpSession session
    ) {
        if (bindingResult.hasErrors()) {
            String message = bindingResult.getFieldErrors().get(0).getDefaultMessage();
            return ResponseEntity.status(HttpStatus.BAD_REQUEST)
                .body(Map.of("status", "error", "message", message));
        }

        Object sessionUserObj = session.getAttribute("user");
        if (!(sessionUserObj instanceof Long sessionUserId) || !sessionUserObj.equals(request.userId)) {
            return ResponseEntity.status(HttpStatus.FORBIDDEN)
                .body(Map.of("status", "error", "message", "Unauthorized user"));
        }

        ... (6줄 생략)
    }

```

수정 가이드: 비밀번호 변경 대상은 요청 바디가 아니라 서버 세션의 인증 주체로 강제해야 합니다. 최소한 세션 사용자와 요청 `userId` 일치 여부를 검증하고, 가능하면 아예 클라이언트가 `userId`를 보내지 않도록 설계하여 서버가 세션에서 직접 대상 계정을 결정해야 합니다.

ISMS-P: ISMS-P 2.6.1(접근통제) 관점에서 사용자 식별정보를 기준으로 한 권한 검증이 누락되어 타인의 계정정보 변경을 허용할 수 있습니다. 또한 인증정보 변경 기능에서 대상 식별자 검증이 부재하므로 계정관리 통제가 미흡한 상태입니다.

AI Audit Trail (추론 과정)

[세션 로그인 값과 요청 바디 값의 권한 기준을 분리해 검토] →
 [``change-pw``에서 대상 `userId`가 요청값으로 결정됨을 확인] → [세션 사용자와 대상 계정 일치 검증 누락을 확인] → [타 계정 비밀번호 변경 가능한 IDOR/권한우회 가능성 도출] → [ISMS-P 접근통제 및 계정관리 통제 미흡으로 매핑]

6. ID 찾기/비밀번호 찾기 기능에서 계정 열거 및 임시 비밀번호 평문 노출

| **Type:** Logic Bypass | **File:**

mini_project5\book\src\main\java\com\aivle0102\book\controller\AuthController

`/user/find-id`는 이름과 전화번호만으로 존재하는 경우 이메일을 직접 반환하고, `/user/find-pw`는 개인정보 일치 시 임시 비밀번호를 응답 본문에 평문

으로 내려줍니다. 이 설계는 계정 존재 여부를 쉽게 추측하게 하며, 임시 비밀번호가 전송 중/로그/클라이언트 저장소에 노출되면 곧바로 계정 탈취로 이어질 수 있습니다. 이는 인증 보조 기능이 오히려 비즈니스 로직상 인증 우회 통로가 되는 사례입니다.

공격 시나리오: 공격자는 이름과 전화번호를 조합해 /find-id를 반복 호출하며 특정 인물의 이메일 존재 여부를 확인할 수 있다. 이후 /find-pw에 여러 조합을 시도해 성공 응답에서 임시 비밀번호를 획득하면, 별도 인증 절차 없이 해당 계정으로 로그인해 점유할 수 있다.

취약 코드:

```
if (user == null) {
    return ResponseEntity.status(404).body(Map.of("message", "일치하는 정보 없음"));
}
return ResponseEntity.ok(Map.of("email", user.getEmail()));
...
String tempPassword = userService.resetPassword(email, name, phone);
data.put("tempPassword", tempPassword);
```

수정 코드:

```
// 예시: 비밀번호 재설정은 토큰 기반으로 전환
@PostMapping("/find-pw")
public ResponseEntity<?> findPassword(@RequestBody Map<String, String> request) {
    userService.issuePasswordResetToken(request.get("email"), request.get("token"));
    return ResponseEntity.ok(Map.of(
        "status", "success",
        "message", "입력하신 정보가 일치하면 재설정 안내가 전송됩니다."
    ));
}
```

수정 가이드: ID 찾기/비밀번호 찾기는 성공 여부와 계정 식별 정보를 직접 노출하지 않도록 응답을 통일하고, 임시 비밀번호를 평문으로 반환하지 말아야 합니다. 비밀번호 재설정은 1회성 만료 토큰과 별도 인증 채널(이메일/SMS)로 처리하고, 레이트리밋과 실패 횟수 제한을 추가해야 합니다.

ISMS-P: ISMS-P 2.7(암호화 적용) 및 계정관리 관점에서 인증정보(임시 비밀번호)를 응답 본문으로 전달하는 것은 저장·전송 중 보호 원칙에 위배될 수 있습니다. 참조 3의 사례 4처럼 비밀번호 찾기/변경 구간의 보호가 누락되면 개인정보 및 인증정보 유출 위험이 발생합니다.

AI Audit Trail (추론 과정)

[계정 복구 기능의 응답 데이터 흐름을 확인] → [`find-id`가 이메일 존재 여부를 직접 반환함을 확인] → [`find-pw`가 임시 비밀번호를 평문 응답으로 반환함을 확인] → [계정 열거 및 인증정보 노출로 이어지는

비즈니스 로직 결함 도출] → [ISMS-P 2.7 암호화·전송 보호 및 계정관리 통제 미흡으로 매핑]

7. 비밀번호를 평문 비교 조건으로 사용하는 로그인 인증 결함

| **Type:** Logic Bypass | **File:** mini_project5/book/src/main/java/com/aivle0102/book/service/impl/LoginServiceImpl.java

로그인 로직이 `findByEmailAndPassword(email, pw)`로 DB에서 이메일과 비밀번호를 그대로 매칭해 사용합니다. 이 구조는 비밀번호를 서버에서 해시 검증하는 방식이 아니라, 저장 방식이 평문이거나 약한 알고리즘일 경우 그대로 인증 우회/계정 탈취로 이어질 수 있습니다. 특히 인증용 비밀번호를 조회 조건으로 직접 사용하는 방식은 비밀번호 유출 시 즉시 재사용 공격에 취약합니다.

공격 시나리오: 공격자가 사전 유출된 비밀번호나 추측한 값으로 로그인 요청을 반복하면, DB에 동일한 문자열이 저장된 계정은 즉시 인증됩니다. 비밀번호가 평문 또는 취약한 암호화로 저장된 경우 DB 유출 시 대량 계정 탈취로 확장됩니다.

취약 코드:

```
return userInfoRepository
    .findByEmailAndPassword(email, pw)
    .orElseThrow(() -> new RuntimeException("로그인 실패"));
```

수정 코드:

```
String email = user.getEmail();
String pw = user.getPassword();
UserInfo account = userInfoRepository.findByEmail(email)
    .orElseThrow(() -> new RuntimeException("로그인 실패"));
if (!passwordEncoder.matches(pw, account.getPassword())) {
    throw new RuntimeException("로그인 실패");
}
return account;
```

수정 가이드: 비밀번호는 DB 조회 조건으로 사용하지 말고, 이메일 등 식별자로만 계정을 조회한 뒤 서버에서 BCrypt 같은 안전한 해시 알고리즘으로 검증해야 합니다. 저장 비밀번호도 반드시 단방향 해시로 관리하고, 인증 실패 메시지는 일관되게 유지하세요.

ISMS-P: ISMS-P DBMS 계정 관리(D-08) 및 Unix 계정 관리(U-13) 취지에 부합하지 않음: SHA-256 이상 수준의 안전한 비밀번호 암호화/검증 정책을 적용하지 않고 인증값을 직접 비교하는 구조는 비밀번호 보호를 충분히 보장하지 못함.

AI Audit Trail (추론 과정)

[로그인 인증 흐름에서 사용자 입력이 어떻게 검증되는지 확인] → [비밀번호가 서버 해시 검증이 아닌 DB 조회 조건으로 직접 사용됨을 확인] → [저장 방식이 평문/취약 해시일 경우 인증 우회 및 계정 탈취로 악용 가능성 판단] → [안전한 비밀번호 암호화 알고리즘 사용 요구(D-08, U-13)와 매핑] → [서버 측 `passwordEncoder` 기반 검증으로 변경 필요성 도출]

8. 비밀번호 재설정 로직의 자격 검증 부족으로 인한 계정 탈취 가능성

| **Type:** Logic Bypass | **File:**

`mini_project5\book\src\main\java\com\aivle0102\book\service\impl\UserServ`

`resetPassword()`는 이메일, 이름, 전화번호의 조합만 일치하면 임시 비밀번호를 생성해 반환합니다. 별도의 OTP, 이메일 소유 확인 링크, 기존 인증 세션 검증, 시도 횟수 제한이 없어 개인정보 일부를 아는 공격자가 비밀번호를 재설정할 수 있습니다. 이는 인증/인가가 아닌 추측 가능한 식별정보 기반으로 계정 제어권을 넘겨주는 비즈니스 로직 결함입니다.

공격 시나리오: 공격자가 대상 사용자의 이메일과 이름, 전화번호를 확보한 뒤 `resetPassword` API를 호출하면 임시 비밀번호가 발급됩니다. 이후 해당 임시 비밀번호로 로그인하거나, 비밀번호 변경을 통해 계정을 장악할 수 있습니다. 이름/전화번호는 유출 가능성이 높고 레이트리미트가 없어 반복 시도도 가능합니다.

취약 코드:

```
UserInfo user = userInfoRepository
    .findByEmailAndNameAndPhone(email, name, phone)
    .orElseThrow(() -> new IllegalArgumentException("일치하는

String tempPassword = UUID.randomUUID().toString().substring(0, 8);
user.setPassword(tempPassword);
userInfoRepository.save(user);
return tempPassword;
```

수정 코드:

```
public String resetPassword(String email, String name, String phone, String otpToken) {
    if (!otpService.verify(email, otpToken)) {
        throw new IllegalArgumentException("본인 확인에 실패했습니다.");
    }
    UserInfo user = userInfoRepository.findByEmail(email)
        .orElseThrow(() -> new IllegalArgumentException("일치하는 사용자가 없습니다."));
    String tempPassword = passwordGenerator.generate();
    user.setPassword(passwordEncoder.encode(tempPassword));
    userInfoRepository.save(user);
    return tempPassword;
}
```

수정 가이드: 비밀번호 재설정은 이메일/전화번호 같은 추측 가능한 정보가 아니라, 이메일 링크 또는 OTP와 같은 소유 증명 기반의 단발성 인증으로 전환해야 합니다. 또한 응답 노출을 최소화하고, 시도 횟수 제한과 레이트리미트를 추가해야 합니다. ISMS-P 2.7 암호화 적용 관점에서도 비밀번호는 안전한 일방향 암호화와 안전한 인증 절차가 함께 적용되어야 합니다.

ISMS-P: ISMS-P 2.7 암호화 적용: 개인정보 및 주요정보의 저장·전달 시 안전한 암호화 및 적절한 인증 절차 수립 필요. 또한 안내서 결함사례(비밀번호 찾기/변경 구간에서 보호조치 누락)와 유사하게, 본인 확인이 약한 상태에서 비밀번호 재설정이 가능하여 계정정보 보호 요구를 충족하지 못함.

AI Audit Trail (추론 과정)

[resetPassword의 입력값(email/name/phone)만으로 계정 식별하는지 확인] → [본인 소유 증명(OTP/이메일 링크/세션 검증) 부재를 확인] → [추측 가능한 개인정보 조합만으로 비밀번호 재설정 가능하므로 **Logic Bypass**로 판정] → [계정 제어권 탈취로 이어지는 실질적 공격 시나리오를 도출] → [ISMS-P 2.7 암호화 적용 및 비밀번호 찾기 보호조치 미흡으로 매핑]

9. 비밀번호 변경 로직의 사용자 소유권 검증 부재로 인한 권한 우회 가능성

| **Type:** IDOR | **File:**

mini_project5\book\src\main\java\com\aiivle0102\book\service\impl\UserService

changePassword()는 userId를 직접 입력받아 해당 계정을 조회한 뒤 현재 비

비밀번호만 맞으면 변경합니다. 이 메서드 자체에는 '현재 인증된 사용자'와 '대상 userId'의 일치 여부를 검증하는 로직이 없어, 상위 계층에서 강제하지 않으면 타인의 계정을 대상으로 변경 요청을 시도할 수 있습니다. 서비스 레벨에서 소유권 검증이 보장되지 않는 점이 핵심 결함입니다.

공격 시나리오: 공격자가 다른 사용자의 userId를 알아낸 뒤 해당 계정의 현재 비밀번호를 알고 있거나 유출된 경우, 자신의 세션이 아니더라도 해당 계정의 비밀번호를 변경할 수 있습니다. 컨트롤러에서 별도 검증이 없다면 전형적인 IDOR/권한 우회가 됩니다.

취약 코드:

```
UserInfo user = userInfoRepository.findById(userId)
    .orElseThrow(() -> new IllegalArgumentException("사용자를 찾을 수

if (!user.getPassword().equals(currentPassword)) {
    throw new IllegalArgumentException("현재 비밀번호가 일치하지 않습니다.")
}

if (currentPassword.equals(newPassword)) {
    throw new IllegalArgumentException("새 비밀번호는 이전 비밀번호와 달라야

}

user.setPassword(newPassword);
userInfoRepository.save(user);
```

수정 코드:

```
public void changePassword(Long currentUserId, Long targetUserId, String
    if (!currentUserId.equals(targetUserId)) {
        throw new SecurityException("본인 계정만 변경할 수 있습니다.");
    }
    UserInfo user = userInfoRepository.findById(targetUserId)
        .orElseThrow(() -> new IllegalArgumentException("사용자를 찾을 수
    if (!passwordEncoder.matches(currentPassword, user.getPassword()))
        throw new IllegalArgumentException("현재 비밀번호가 일치하지 않습니
    }
    user.setPassword(passwordEncoder.encode(newPassword));
    userInfoRepository.save(user);
}
```

수정 가이드: 비밀번호 변경은 반드시 인증 주체와 대상 계정의 소유권 일치를 확인해야 합니다. 서비스 레벨에서 currentUserId를 강제하거나 SecurityContext에서 현재 사용자 ID를 직접 가져와 비교해야 하며, controller 단에서만 막지 말고 도메인 로직에서도 재검증하는 것이 안전합니다. 비밀번호는 평문 비교 대신 안전한 해시 검증으로 처리해야 합니다.

ISMS-P: ISMS-P 2.7 암호화 적용(비밀번호 처리) 및 접근통제 원칙 위반 소지. 안내서의 계정정보 보호 요구와 함께, 인증된 사용자만 자신의 계정 변경이 가능

하도록 통제해야 하는 내부 접근통제 요구에 부합하지 않음.

AI Audit Trail (추론 과정)

[changePassword가 userId 기반으로 대상 계정을 직접 선택하는지 확인] → [인증 주체 (SecurityContext/currentUser)와 대상 userId의 일치 검증 부재를 확인] → [현재 비밀번호만 알면 다른 계정도 변경 가능한 구조인지 판단] → [권한 우회 가능한 IDOR로 분류] → [비밀번호 보호 및 접근통제 관점에서 ISMS-P 2.7과 연계]

10. 외부 API 키 및 모델 파라미터를 클라이언트 입력에 의존하는 정책 우회 가능성

| **Type:** Logic Bypass | **File:**

mini_project5\book\frontend\src\app\api\generateCover\route.js

해당 라우트는 요청 본문에서 받은 apiKey를 그대로 OpenAI 호출의 Authorization 헤더에 사용하고, model도 서버 검증 없이 전달합니다. 즉, 서버가 통제해야 할 인증/과금/허용모델 정책이 클라이언트 입력에 의해 결정되어 비즈니스 정책 우회가 가능합니다. 이는 단순 입력 검증 문제가 아니라 신뢰 경계가 깨진 설계 결함입니다.

공격 시나리오: 공격자가 임의의 apiKey와 고비용/비허용 model 값을 넣어 요청하면, 서버는 이를 검증 없이 외부 API로 중계합니다. 이로 인해 서비스가 의도한 키 통제·과금 통제·모델 사용 정책을 우회할 수 있고, 반복 호출 시 외부 API 비용/자원 오남용이 발생할 수 있습니다.

취약 코드:

```
const { apiKey, model, prompt } = await req.json();
...
"Authorization": `Bearer ${apiKey}`,
...
body: JSON.stringify({
  model: model || "dall-e-3",
  prompt,
  n: 1,
  size: "1024x1024"
})
```

수정 코드:

```
const allowedModels = new Set(["dall-e-3"]);

export async function POST(req) {
  const user = await authenticate(req);
  if (!user) return NextResponse.json({ error: "Unauthorized" }, { status: 401 });

  const { model, prompt } = await req.json();
  if (!prompt || prompt.length > 1000) {
    return NextResponse.json({ error: "Invalid prompt" }, { status: 400 });
  }
  if (!allowedModels.has(model || "dall-e-3")) {
    return NextResponse.json({ error: "Model not allowed" }, { status: 403 });
  }

  const openaiRes = await fetch("https://api.openai.com/v1/images/genera... (13줄 생략)
```

수정 가이드: 클라이언트가 API 키를 주입하지 못하도록 서버 환경변수로 외부 API 키를 고정하고, 허용 모델을 allowlist로 제한해야 합니다. 또한 인증된 사용자만 호출 가능하도록 서버 측 인증/인가를 추가하고, 요청 횟수 제한 및 prompt 길이 제한을 적용하여 정책 우회와 비용 오남용을 방지해야 합니다.

ISMS-P: ISMS-P의 최소권한 원칙 및 세밀한 접근제어 요구에 위배될 수 있으며, 제공된 참고문헌의 OT 제로트러스트 개요(리소스 분류 및 최소 권한 부여, 논리 경계 생성 및 세션 단위 접근 허용, 모든 상태 모니터링 및 로그)와 주요정보통신 기반시설 상세가이드의 웹 비즈니스 로직 점검 항목(예상된 프로세스 흐름 검토, 인증이 필요한 주요 정보 페이지의 권한 검증, 서버 사이드 접근 통제 구현) 취지에 반합니다.

AI Audit Trail (추론 과정)

[요청 본문에서 **apiKey/model/prompt**를 직접 받는 구조를 확인] → [apiKey가 외부 API 인증 경계에 그대로 주입되는지 추적] → [서버 측 인증/인가, 허용모델 검증, 쿼터 통제 부재를 확인] → [클라이언트 입력이 정책과 과금에 영향을 주는 비즈니스 로직 우회로 판단] → [제로트러스트의 최소권한·논리경계·지속적 검증 원칙 및 웹 권한 검증 가이드와 매핑]

11. 임시 비밀번호를 API 응답과 화면에 평문으로 노출

| **Type:** Logic Bypass | **File:**

mini_project5\book\frontend\src\app\find\password\page.js

비밀번호 찾기 기능이 성공 응답에서 `result.data.tempPassword`를 그대로 받아 화면에 출력합니다. 이 구조는 임시 비밀번호가 브라우저 UI, 네트워크 구간, 클라이언트 로그/스크린샷/확장프로그램 등 다양한 경로로 노출될 수 있어 계정 탈취 표면을 확대합니다. 비밀번호 찾기 절차는 일반적으로 일회성 토큰/인증 링크/OTP 기반으로 전환해야 하며, 임시 비밀번호를 응답 바디로 반환하는 방식은 지양해야 합니다. ISMS-P 2.7 암호화 적용의 사례 4(비밀번호 찾기 등 개인정보 전송 구간 암호화 누락)와 사례 5(인증키/비밀번호의 평문 저장·노출 취지)에 부합하는 위험으로 볼 수 있습니다.

공격 시나리오: 공격자는 비밀번호 찾기 폼을 이용해 특정 계정의 가입 정보와 일치하는 값을 제출한 뒤, 서버가 임시 비밀번호를 응답에 포함하는 구조를 악용해 그 비밀번호를 즉시 획득할 수 있습니다. 이후 해당 임시 비밀번호로 로그인하여 계정을 장악할 수 있으며, 응답이 클라이언트에 남기 때문에 브라우저 캐시/화면 캡처/개발자도구를 통한 추가 노출도 가능합니다.

취약 코드:

```
const result = await response.json();
...
setTempPassword(result.data.tempPassword);
...
{tempPassword && (
  <div style={styles.resultBox}>
    <p style={{ marginBottom: 6 }}>발급된 임시 비밀번호</p>
    <p style={styles.tempPw}>{tempPassword}</p>
  </div>
)}
```

수정 코드:

```
// 서버는 tempPassword를 응답하지 않고, 일회성 토큰/OTP만 반환
const response = await fetch('/api/user/find-pw', { method: 'POST', headers: { 'Content-Type': 'application/json' } });
const result = await response.json();
if (!response.ok || result.status !== 'success') {
  alert(result.message || '비밀번호 찾기에 실패했습니다.');
```

수정 가이드: 임시 비밀번호를 클라이언트에 직접 반환하지 말고, 이메일/문자 기반의 일회성 인증 링크 또는 OTP를 사용하도록 변경해야 합니다. 비밀번호 재설정에는 별도 인증 완료 후 서버에서 수행하고, 화면에는 민감정보를 표시하지 않도록 해야 합니다. 또한 전송 구간 TLS를 강제하고, 응답/로그에 비밀번호 또는 재설정 토큰이 남지 않도록 점검해야 합니다.

ISMS-P: ISMS-P 2.7 암호화 적용 위반 가능성: 비밀번호 찾기/변경 과정에서 민감정보가 클라이언트에 평문 노출됨. 참조 3의 사례 4(비밀번호 찾기 등 개인정보 전송 구간 암호화/보호 조치 누락) 및 사례 5(비밀번호/인증키의 평문 저장·노출)

출)에 해당하는 위험이 존재함.

AI Audit Trail (추론 과정)

[비밀번호 찾기 화면의 데이터 흐름 식별] → [성공 응답의 `tempPassword`를 UI에 그대로 렌더링하는 구조 확인] → [평문 임시 비밀번호 노출로 계정 탈취 가능성 도출] → [ISMS-P 2.7 암호화 적용 및 사례 4.5와 매핑] → [클라이언트 노출 제거 및 일회성 인증 기반 재설계 필요성 판단]

12. 세션 생성·만료 정책이 보이지 않는 불충분한 세션 관리 가능성

| **Type:** Logic Bypass | **File:** mini_project5/book/src/main/java/com/aivle0102/book/service/impl/LoginServiceImpl.java

해당 서비스는 `HttpSession`을 import하지만 실제 로그인 성공 시 세션 생성, 사용자 식별자 저장, 만료 시간 설정이 보이지 않습니다. 이 코드만으로 단정할 수는 없지만, 로그인 성공 이후 세션 바인딩이 다른 계층에 없다면 인증 상태가 적절히 유지되지 않아 타 사용자 세션 재사용, 인증 컨텍스트 혼선, 접근 통제 누락으로 이어질 수 있습니다.

공격 시나리오: 공격자가 로그인 성공 후에도 세션이 적절히 갱신·만료되지 않는 환경을 이용하면, 탈취한 세션을 장기간 재사용하거나 로그인 상태가 잘못 유지되는 취약점을 악용할 수 있습니다. 또한 세션 고정 방어가 없다면 인증 전 세션이 인증 후에도 재사용될 수 있습니다.

취약 코드:

```
import jakarta.servlet.http.HttpSession;
...
@Override
public UserInfo getUser(UserInfo user) {
    ...
    return userInfoRepository
        .findByEmailAndPassword(email, pw)
        .orElseThrow(() -> new RuntimeException("로그인 실패"));
}
```

수정 코드:

```

@Override
public UserInfo getUser(UserInfo user, HttpSession session) {
    String email = user.getEmail();
    String pw = user.getPassword();
    UserInfo account = userInfoRepository.findByEmail(email)
        .orElseThrow(() -> new RuntimeException("로그인 실패"));
    if (!passwordEncoder.matches(pw, account.getPassword())) {
        throw new RuntimeException("로그인 실패");
    }
    session.invalidate();
    HttpSession newSession = session;
    newSession.setAttribute("LOGIN_USER", account.getId());
    newSession.setMaxInactiveInterval(30 * 60);
    return account;
}

```

수정 가이드: 로그인 성공 시 세션을 재생성하고 사용자 식별자를 서버 세션에 저장해야 합니다. 또한 세션 만료 시간과 자동 로그아웃 정책을 명시해 세션 재사용을 방지하세요. JWT를 사용할 경우에도 만료, 서명, 키 관리 정책을 함께 강제해야 합니다.

ISMS-P: ISMS-P 웹 애플리케이션 불충분한 세션 관리(16) 취지 위반 가능성: 세션 ID 발급/만료/고정 방지 정책이 코드상 확인되지 않음.

AI Audit Trail (추론 과정)

[로그인 성공 후 사용자 상태 유지 방식이 코드에 존재하는지 확인] → [HttpSession import는 있으나 실제 세션 생성·만료 설정이 없음 확인] → [세션 고정/재사용 방지 및 만료 정책 부재 시 인증 상태 악용 가능성 판단] → [ISMS-P 웹 취약점 16(불충분한 세션 관리)와 매핑] → [세션 재생성 및 만료 설정 도입 필요성 도출]

13. 회원가입 이메일 중복검사와 저장 사이 원자성 부족으로 중복 계정 생성 가능

| **Type:** Race Condition | **File:**

mini_project5\book\src\main\java\com\aivle0102\book\service\impl\UserServ

signUp()은 먼저 findByEmail()로 중복을 확인한 뒤 save()를 수행합니다. 이 패턴은 동시 요청에서 TOCTOU 경합이 발생할 수 있어, 동시에 두 요청이 들어오면 둘 다 '없음'으로 판단하고 저장될 수 있습니다. DB 유니크 제약이 코

드상 보이지 않아 중복 가입을 완전히 방지하지 못합니다.

공격 시나리오: 공격자가 동일 이메일로 매우 짧은 간격의 가입 요청을 여러 번 동시에 전송하면, 중복 검사 통과 후 여러 계정이 생성될 수 있습니다. 이후 비밀번호 재설정, 계정 식별, 감사 추적에서 혼선이 발생합니다.

취약 코드:

```
userInfoRepository.findByEmail(request.getEmail())
    .ifPresent(user -> {
        throw new IllegalStateException("이미 가입된 이메일입니다.");
    });

UserInfo user = new UserInfo();
user.setEmail(request.getEmail().trim().toLowerCase());
...
return userInfoRepository.save(user);
```

수정 코드:

```
@Transactional
public UserInfo signUp(UserSignUpRequest request) {
    UserInfo user = new UserInfo();
    user.setEmail(request.getEmail().trim().toLowerCase());
    ...
    try {
        return userInfoRepository.save(user);
    } catch (DataIntegrityViolationException e) {
        throw new IllegalStateException("이미 가입된 이메일입니다.");
    }
}
```

수정 가이드: 중복 방지는 애플리케이션의 사전 조회가 아니라 DB의 UNIQUE 제약으로 보장해야 합니다. 이후 예외를 사용자 친화적으로 변환하는 방식이 안전합니다. 필요하다면 가입 요청 레이트리밋도 추가해 동시성 악용을 줄이세요.

ISMS-P: ISMS-P 관점에서 계정정보 무결성 및 접근통제 실패로 이어질 수 있으며, 중복 계정 생성은 인증/추적의 신뢰성을 저해합니다. 직접적인 암호화 항목은 아니지만 계정 관리 통제 미흡으로 해석될 수 있습니다.

AI Audit Trail (추론 과정)

[**signUp**의 이메일 중복검사를 선조회 방식으로 확인] → [조회와 저장 사이에 동시 요청이 겹칠 경우 경합 가능성 판단] → [DB **UNIQUE** 제약 부재 시 중복 계정 생성 가능성 도출] → [TOCTOU 기반 **Race Condition**으로 분류] → [계정 무결성 훼손으로 인한 관리 통제 미흡으로 정리]